

**Prova 2**

- 1) Escreva uma função que recebe como parâmetro uma string contendo o nome completo de uma pessoa. A função deve imprimir as iniciais dessa pessoa.

As iniciais são formadas pela primeira letra de cada nome, sendo que todas deverão aparecer em maiúsculas na saída do programa. Note que os conectores *e, do, da, dos, das, de, di, du* não são considerados nomes e, portanto, não devem ser considerados para a obtenção das iniciais. As iniciais devem ser impressas em maiúsculas, ainda que o nome seja entrado todo em minúsculas.

**Exemplo:**

```
Entrada 1: s = "Maria das Gracas Pimenta"
Saída 1: MGP
Entrada 2: s = "Joao Carlos dos Santos"
Saída 2: JCS
Entrada 3: s = "pedro pereira teixeira"
Saída 3: PPT
Limitações: Considere que o nome passado como parâmetro
tem, no máximo, 5 iniciais
```

- 2) Faça uma função que recebe duas strings (s1 e s2) e coloca em s2 todas os caracteres da string s1 que não sejam vogais. Retorne o tamanho da string s2.

**Exemplo:**

```
Entrada 1: s1 = "abacate amarelo"
Saída 1: s2 = "bct mrl"
Entrada 2: s1 = "take my breath away"
Saída 2: s2 = "tk my brth wy"
```

- 3) Em sala, trabalhamos com a struct `Restaurante`, onde um restaurante é identificado pelo nome, o tipo de comida (brasileira, chinesa, francesa, italiana, japonesa, etc.) e uma nota para a cozinha (entre 0 e 5). Além disso, vamos adicionar um campo extra para armazenar a distância (em Km) do restaurante até a casa do cliente que fará o pedido nesse estabelecimento. A estrutura proposta está adicionada no projeto.

Faça uma função que recebe um vetor de valores do tipo `Restaurante`, o número de restaurantes cadastrados e **imprima** qual é o nome do melhor (mais alta nota) restaurante de comida chinesa em um raio de 3Km de distância da casa do cliente. Caso não exista, retorne a mensagem "Não existe restaurante chinês em um raio de 3Km".

**Prova 2****Exemplo:**

```
Entrada 1: Sabores, brasileira, 4.5, 3.1
           Jinchuan, chinesa, 4.6, 2
           China in Cube, chinesa, 4.9, 2.1
           Baka, japonesa, 5, 3.1
           Don Ramon, mexicana, 4.2, 1.2
           Game of Litrones, bebidas, 4.9, 0.3
           Yin Yang, chinesa, 5, 4
           Siriguela, frutos do mar, 4.9, 7
numRest = 8
Saída 1: Chine in Cube
Entrada 2: Sabores, brasileira, 4.5, 3.1
           Jinchuan, chinesa, 4.6, 2
           China in Cube, chinesa, 4.9, 2.1
           Baka, japonesa, 5, 3.1
           Don Ramon, mexicana, 4.2, 1.2
           Game of Litrones, bebidas, 4.9, 0.3
           Yin Yang, chinesa, 5, 1
           Siriguela, frutos do mar, 4.9, 7
numRest = 8
Saída 2: Yin Yang
Entrada 3: Sabores, brasileira, 4.5, 3.1
           Don Ramon, mexicana, 4.2, 1.2
           Game of Litrones, bebidas, 4.9, 0.3
NumRest = 3
Saída 3: Não existe restaurante chinês em um raio de 3Km
```

- 4) A estrutura `Compromisso` representa um compromisso marcado em um sistema de agenda. Um compromisso contém: o título, que é uma string, o dia e um horário de início e término, ambos do tipo `Horario`. O tipo `Dia` armazena os campos dia (1 a 30), mês (1 a 12) e ano (1500 a 4000). O tipo `Horario` armazena os campos hora (de 0 a 23) e minutos (de 0 a 59). As structs estão adicionadas ao projeto.

Escreva uma função que recebe um `Compromisso` e retorna a quantidade de minutos que ele dura. Considere que compromissos podem começar e terminar em dias diferentes com o término sempre após o início. Para simplificação, considere que todos os meses possuem 30 dias.

**Exemplo:**

```
Entrada 1: inicio = 13:30 dia = 02/03/2024
           fim = 20:20 dia = 02/03/2024
Saída 1: 410
Entrada 2: inicio = 9:00 dia 12/08/2004
           fim = 12:27 dia 13/08/2005
Saída 2: 520047
```



**INSTITUTO FEDERAL**

Santa Catarina

Câmpus Florianópolis

INSTITUTO FEDERAL DE SANTA CATARINA

CAMPUS FLORIANÓPOLIS

TÉCNICO EM ELETRÔNICA

LINGUAGEM DE PROGRAMAÇÃO C

## Prova 2

- 5) O CPF de um cidadão brasileiro é composto por uma série de 11 dígitos, sendo os 8 primeiros gerados aleatoriamente, o nono dígito indica a região fiscal onde o documento foi emitido e os últimos 2 são o dígito verificador, gerado por um algoritmo que opera sob dígitos anteriores.

As regiões fiscais são:

0 - RS

1 - DF, GO, MT, MS e TO;

2 - AC, AP, AM, PA, RO e RR;

3 - CE, MA e PI;

4 - AL, PB, PE e RN;

5 - BA e SE;

6 - MG;

7 - ES e RJ;

8 - SP;

9 - PR e SC;

Faça uma função que recebe uma lista de strings (matriz de caracteres) contendo vários CPF's, no formato DDD.DDD.DDD-VV, a quantidade de CPF's e **retorne** quantos documentos foram emitidos nas regiões 8 e 9.

### Exemplo:

```
Entrada 1: strings = {"123.456.788-10", "987.654.321-09", "456.123.789-08", "789.321.456-07", "246.135.879-06", "951.357.864-05", "123.789.458-04", "789.456.123-03", "369.852.741-02", "147.258.369-01"}
```

```
Saída 1: 5
```

```
Entrada 2: "123.456.789-01", "987.654.321-02", "246.813.579-03", "135.792.468-04", "864.209.753-05", "975.318.624-06", "528.741.963-07", "369.480.217-08", "741.259.836-09", "852.963.074-10"
```

```
Saída 2: 3
```